

LABORATORIO 15

El caso Contraloría

Guía para leer mientras trabajas el cuaderno. Este laboratorio es el examen, así que la guía trae las respuestas dentro de cada paso. Lee primero la introducción y el oficio, intenta cada pregunta en el cuaderno y recién después abre su sección.

Introducción

Antes de leer. Este laboratorio es el examen del curso. Las respuestas a las cuatro preguntas del oficio están dentro de los pasos 2 a 5 de esta guía. Si quieres aprovecharlo, lee la introducción y el paso 1, ejecuta el paso 0 en el cuaderno sin leer su sección, intenta cada pregunta tú solo y recién entonces lee la sección que le corresponde.

1 · El tema

Llega un oficio de Contraloría preguntando por un contribuyente, y tienes que reconstruir su historia y responder. No hay nada nuevo que aprender. Todo lo que hace falta ya lo viste en el curso, sobre todo en los laboratorios 00 y 01, y lo que se evalúa es reconocer qué usar ante una pregunta que no viene con instrucciones.

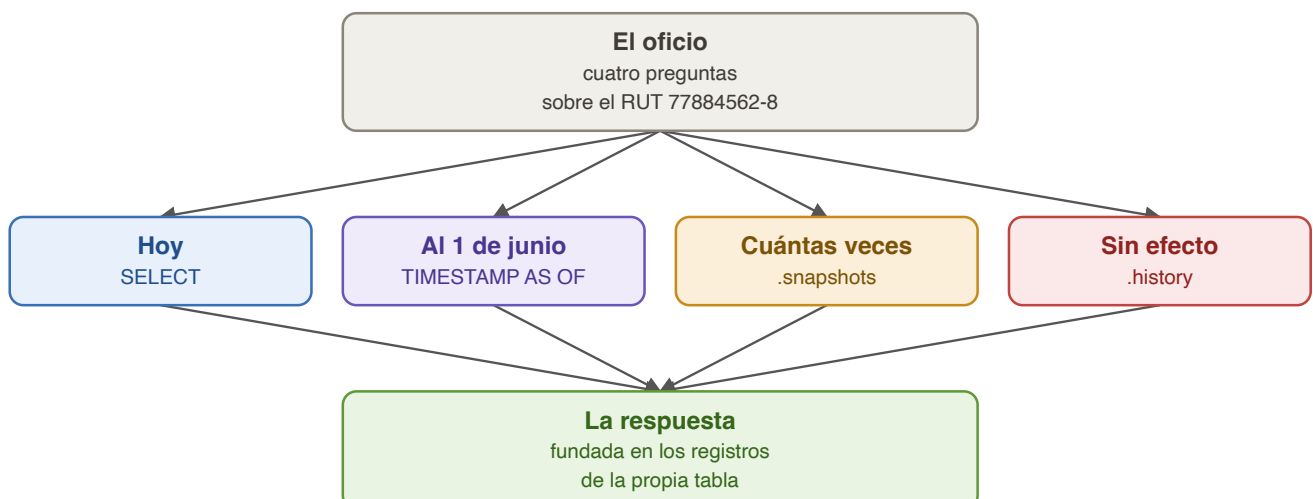
2 · El problema, en el almacén

Un inspector llega al almacén y pregunta por un cliente. Quiere saber qué dice hoy su renglón, qué decía en una fecha anterior, cuántas veces se corrigió la libreta y si alguna corrección se deshizo después. Y quiere que la respuesta salga de la libreta, no de lo que recuerde el almacenero.

3 · El problema en términos técnicos

En una base de datos común, cuando se corrige una fila el valor anterior desaparece. Para responder un oficio así hay que buscar en respaldos, reconstruir la tabla de una fecha pasada y firmar que eso era lo que había. Puede tomar días y descansa en la memoria de una persona. Y si nadie activó un registro de auditoría a tiempo, no hay con qué responder.

4 · El diagrama



El oficio (gris). Contraloría pregunta cuatro cosas sobre Araucaria Ferretería SpA, RUT 77884562-8.

Hoy (azul). El segmento que tiene hoy se responde con un `SELECT` sobre la tabla.

Al 1 de junio (morado). El segmento en una fecha pasada se pregunta con `TIMESTAMP AS OF`, que es el viaje en el tiempo por fecha.

Cuántas veces (amarillo). Las modificaciones sobre el padrón están en la vista `.snapshots`, una fila por página.

Sin efecto (rojo). Si alguna modificación se dejó sin efecto lo dice la vista `.history`, con la columna `is_current_ancestor`.

La respuesta (verde). Las cuatro respuestas salen de la tabla misma. Nadie activó un registro de auditoría.

5 · La solución

En el almacén, la libreta no borra páginas. Cada corrección es una página nueva, fechada, y la libreta anota cuál página mandó en cada momento. Con eso el almacenero puede mostrarle al inspector el renglón de hoy, el de cualquier página anterior, la lista de páginas y cuáles quedaron fuera del camino cuando se volvió atrás.

En Iceberg cada escritura deja un snapshot con su fecha, y la tabla guarda dos listas. `.snapshots` lista todas las páginas que se escribieron. `.history` lista cuándo cada página pasó a ser la vigente, y marca con `is_current_ancestor` si sigue en el camino que llega a la de hoy. Con `VERSION AS OF` y `TIMESTAMP AS OF` se lee la tabla en cualquier página que exista.

6 · Los pasos

- **Paso 0.** Diez celdas de montaje que arman el padrón con una historia que no viste.
- **Paso 1.** El oficio de Contraloría.
- **Paso 2.** Primera pregunta, el segmento de hoy.
- **Paso 3.** Segunda pregunta, el segmento al 1 de junio de 2026.
- **Paso 4.** Tercera pregunta, las modificaciones y su fecha.
- **Paso 5.** Cuarta pregunta, si alguna se dejó sin efecto.
- **Paso 6.** La respuesta al oficio.

Si repites el laboratorio, el paso 0 borra la tabla y la vuelve a armar. Si prefieres partir limpio, antes corre el comando de abajo, desde la carpeta del repositorio.

```
bash
bin/reiniciar-lab.sh 15
```

Paso 0 · Montaje

Si vas a hacer el examen, ejecuta las diez celdas del cuaderno sin leer esta sección, lee el paso 1 y vuelve aquí cuando hayas respondido. Esta sección cuenta la historia que el montaje arma, y esa historia es la respuesta.

Celda 0.1 · Entrar a tu espacio

La pregunta. ¿En qué espacio va a quedar la tabla del caso?

Por qué ahora. Todas las celdas que siguen nombran la tabla sin espacio adelante, así que primero hay que estar en `mi_espacio`.

En el almacén. Es pararte frente a tu propio estante.

La sentencia, parte por parte.

- `%%sql` le dice al cuaderno que la celda es SQL.
- `-- Celda 0.1` es un comentario, que el motor ignora.
- `USE mi_espacio` deja a `mi_espacio` como el espacio por omisión.

```
sql
%%sql
-- Celda 0.1
USE mi_espacio
```

Lo que sale en pantalla.

```
text
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
26/09/25 18:11:11 WARN NativeCodeLoader: Unable to load native-hadoop library for your
platform... using builtin-java classes where applicable
Listo. La sentencia se ejecutó.
```

Cómo se lee. La sentencia se ejecutó. Las tres primeras líneas son avisos de Spark al arrancar, no errores.

- `Setting default log level to "WARN"` dice que Spark va a mostrar solo los avisos y los errores.
- `To adjust logging level use sc.setLogLevel(newLevel)` explica cómo cambiar ese nivel. `sc` es el contexto de Spark. SparkR es la versión de Spark para el lenguaje R, que aquí no se usa.
- `WARN NativeCodeLoader: Unable to load native-hadoop library` dice que Spark no encontró una biblioteca de Hadoop para este sistema operativo y usa la versión en Java. Funciona igual. `26/09/25 18:11:11` es la fecha y la hora del aviso, en UTC.

Celda 0.2 · Partir de cero

La pregunta. ¿Quedó una tabla del caso de una corrida anterior?

Por qué ahora. El montaje crea la tabla desde cero, y si ya existe la creación falla.

En el almacén. Es sacar del estante la libreta de un intento anterior.

La sentencia, parte por parte.

- `DROP TABLE` borra la tabla.
- `IF EXISTS` evita el error si no existe.
- `contribuyentes_lab15` es la tabla del caso.

```
sql
%%sql
-- Celda 0.2
DROP TABLE IF EXISTS contribuyentes_lab15
```

Lo que sale en pantalla.

```
text
Listo. La sentencia se ejecutó.
```

Cómo se lee. Listo. La tabla anterior, si había, ya no está.

Celda 0.3 · Crear el padrón

La pregunta. ¿Qué forma tiene el padrón?

Por qué ahora. Antes de cargar contribuyentes hay que crear la tabla.

En el almacén. Es abrir una libreta nueva con tres casillas por renglón.

La sentencia, parte por parte.

- `CREATE TABLE contribuyentes_lab15` crea la tabla.
- `rut STRING`, `razon_social STRING` y `segmento STRING` son las tres columnas, las tres de texto.
- `USING iceberg` hace que sea una tabla Iceberg, con su historia de páginas.

```
sql
%%sql
-- Celda 0.3
CREATE TABLE contribuyentes_lab15 (
    rut          STRING,
    razon_social STRING,
    segmento     STRING
) USING iceberg
```

Lo que sale en pantalla.

```
text
Listo. La sentencia se ejecutó.
```

Cómo se lee. La tabla existe y está vacía. Todavía no tiene ninguna página.

Celda 0.4 · La carga inicial del padrón

La pregunta. ¿Con qué contribuyentes parte el padrón?

Por qué ahora. Esta carga es la primera página de la libreta, y es la más antigua que va a existir.

En el almacén. Es anotar los primeros diez clientes.

La sentencia, parte por parte.

- `INSERT INTO contribuyentes_lab15 VALUES` agrega filas escritas a mano.
- Cada paréntesis es una fila con el RUT, la razón social y el segmento. Son diez contribuyentes.
- Araucaria Ferretería SpA, RUT `77884562-8`, entra como `PEQUENA`. Es el contribuyente del oficio.

```
sql
%%sql
-- Celda 0.4
INSERT INTO contribuyentes_lab15 VALUES
  ('77746521-K', 'Pehuen Logistica EIRL', 'MICRO'),
  ('77884562-8', 'Araucaria Ferreteria SpA', 'PEQUENA'),
  ('78800840-6', 'Copihue Maquinarias EIRL', 'MEDIANA'),
  ('76171162-8', 'Huemul Alimentos EIRL', 'GRANDE'),
  ('76000262-3', 'Bolder Importadora EIRL', 'MICRO'),
  ('76002248-9', 'Andes Consultores SpA', 'MEDIANA'),
  ('76005135-7', 'Huemul Servicios SpA', 'GRANDE'),
  ('76008516-2', 'Bolder Logistica SpA', 'PEQUENA'),
  ('76011940-7', 'Canelo Servicios EIRL', 'MICRO'),
  ('76015893-3', 'Lenga Consultores Ltda.', 'MEDIANA')
```

Lo que sale en pantalla.

```
text
Listo. La sentencia se ejecutó.
```

Cómo se lee. Entraron las diez filas. Esa escritura dejó la primera página.

Celda 0.5 · Una reclasificación

La pregunta. ¿Qué le pasa a Araucaria después de la carga?

Por qué ahora. Es la primera modificación de su historia.

En el almacén. Es corregir el renglón de Araucaria de pequeña a mediana.

La sentencia, parte por parte.

- `UPDATE contribuyentes_lab15` cambia filas que ya existen.
- `SET segmento = 'MEDIANA'` pone el segmento nuevo.
- `WHERE rut = '77884562-8'` apunta solo a Araucaria.

```
sql
%%sql
-- Celda 0.5
UPDATE contribuyentes_lab15 SET segmento = 'MEDIANA' WHERE rut = '77884562-8'
```

Lo que sale en pantalla.

```
text
Listo. La sentencia se ejecutó.
```

Cómo se lee. Araucaria quedó en `MEDIANA`. La corrección dejó la segunda página.

Celda 0.6 · Tres contribuyentes nuevos

La pregunta. ¿Qué más entra al padrón?

Por qué ahora. Es otra escritura sobre el padrón, que va a quedar en la historia.

En el almacén. Es anotar tres clientes nuevos al final.

La sentencia, parte por parte.

- `INSERT INTO contribuyentes_lab15 VALUES` agrega filas.
- Los tres paréntesis son Ulmo Constructora, Canelo Constructora y Maiten Alimentos.

```
sql
%%sql
-- Celda 0.6
INSERT INTO contribuyentes_lab15 VALUES
    ('76004358-3', 'Ulmo Constructora Ltda.', 'PEQUENA'),
    ('76006049-6', 'Canelo Constructora S.A.', 'MEDIANA'),
    ('76007969-3', 'Maiten Alimentos EIRL', 'GRANDE')
```

Lo que sale en pantalla.

```
text
Listo. La sentencia se ejecutó.
```

Cómo se lee. Entraron las tres filas. Es la tercera página.

Celda 0.7 · Volver a la primera página

La pregunta. ¿Cómo se deja la libreta como estaba en su primera página?

Por qué ahora. Es la parte de la historia que el oficio va a preguntar en su cuarta pregunta, una reversión.

En el almacén. Es declarar que la primera página vuelve a mandar. Las páginas de la reclasificación y de los tres nuevos no se arrancan, quedan en la libreta pero fuera del camino.

Esta celda es de Python y viene escrita. Es una celda auxiliar. Para volver a una página hace falta su número, y `CALL` solo acepta números escritos en la sentencia, no una consulta que los busque. Así que la celda busca el número de la primera página y lo pega dentro del `CALL`. El comentario sobre la Regla 6 del estándar didáctico del curso quiere decir justamente eso, que es una ayuda del montaje y que su forma no se evalúa.

La sentencia, parte por parte.

- Las líneas que empiezan con `#` son comentarios de Python.
- `spark.catalog.currentDatabase()` devuelve el espacio en que estás, `mi_espacio`. Se le pega `".contribuyentes_lab15"` para tener el nombre completo de la tabla en la variable `tabla`.
- `spark.sql("SELECT snapshot_id FROM " + tabla + ".snapshots ORDER BY committed_at LIMIT 1")` pide a la vista de páginas la más vieja. `ORDER BY committed_at` ordena de la más vieja a la más nueva y `LIMIT 1` se queda con la primera.
- `.first()[0]` toma la primera fila del resultado y su primera columna. Ese número queda en la variable `primera`.
- `spark.sql("CALL spark_catalog.system.rollback_to_snapshot('" + tabla + "', " + str(primera) + ")")` arma y ejecuta la reversión. `rollback_to_snapshot` es el procedimiento de Iceberg que vuelve a una página, y recibe el nombre de la tabla entre comillas y el número de la página. `str(primera)` convierte el número en texto para pegarlo.
- `print("Montaje: listo.")` avisa que terminó.

```
python
# Celda 0.7
# Montaje. Vuelve la libreta a su primera pagina.
#
# Va en Python y no en %%sql por una sola razon: 'CALL' solo acepta literales,
# asi que el identificador de la pagina hay que buscarlo antes y pegarlo dentro
# de la sentencia. Es la funcion auxiliar de la Regla 6 del estandar didactico:
# no es una operacion del trabajo del alumno y su sintaxis no se evalua.
tabla = spark.catalog.currentDatabase() + ".contribuyentes_lab15"

primera = spark.sql(
    "SELECT snapshot_id FROM " + tabla + ".snapshots ORDER BY committed_at LIMIT 1"
).first()[0]

spark.sql(
    "CALL spark_catalog.system.rollback_to_snapshot('" + tabla + "', " + str(primera) + ")"
)
print("Montaje: listo.")
```

Lo que sale en pantalla.

```
text
Montaje: listo.
```

Cómo se lee. La reversión se hizo. Desde aquí la tabla dice lo mismo que en la primera página, con Araucaria otra vez en `PEQUENA` y sin los tres nuevos.

Celda 0.8 · Otra reclasificación

La pregunta. ¿Qué le pasa a Araucaria después de la reversión?

Por qué ahora. Es la corrección que va a quedar vigente.

En el almacén. Es corregir el renglón de Araucaria, esta vez a grande.

La sentencia, parte por parte.

- `UPDATE contribuyentes_lab15` cambia filas existentes.
- `SET segmento = 'GRANDE'` pone el segmento nuevo.
- `WHERE rut = '77884562-8'` apunta solo a Araucaria.

```
sql
%%sql
-- Celda 0.8
UPDATE contribuyentes_lab15 SET segmento = 'GRANDE' WHERE rut = '77884562-8'
```

Lo que sale en pantalla.

```
text
Listo. La sentencia se ejecutó.
```

Cómo se lee. Araucaria quedó en `GRANDE`.

Celda 0.9 · Una columna nueva

La pregunta. ¿Cambia la forma del padrón?

Por qué ahora. Es un cambio de esquema en medio de la historia, y aparece en la respuesta de la primera pregunta.

En el almacén. Es agregar una casilla nueva a los renglones, la región. Los renglones que ya estaban la tienen en blanco.

La sentencia, parte por parte.

- `ALTER TABLE contribuyentes_lab15` cambia la definición de la tabla.
- `ADD COLUMN region STRING` agrega la columna `region`, de texto.

```
sql
%%sql
-- Celda 0.9
ALTER TABLE contribuyentes_lab15 ADD COLUMN region STRING
```

Lo que sale en pantalla.

```
text
Listo. La sentencia se ejecutó.
```

Cómo se lee. La tabla tiene una columna más. Este cambio no escribe datos, así que no deja una página nueva.

Celda 0.10 · Una baja

La pregunta. ¿Sale alguien del padrón?

Por qué ahora. Es la última escritura del montaje.

En el almacén. Es tachar el renglón de un cliente que se dio de baja.

La sentencia, parte por parte.

- `DELETE FROM contribuyentes_lab15` borra filas.
- `WHERE rut = '76011940-7'` apunta a Canelo Servicios EIRL.

```
sql
%%sql
-- Celda 0.10
DELETE FROM contribuyentes_lab15 WHERE rut = '76011940-7'
```

Lo que sale en pantalla.

```
text
Listo. La sentencia se ejecutó.
```

Cómo se lee. Canelo Servicios salió del padrón. Con esto termina el montaje.

Paso 1 · El oficio

Este paso no tiene celdas. Es el oficio de Contraloría, y conviene leerlo con cuidado.

La pregunta. ¿Qué pide exactamente Contraloría?

Por qué ahora. Cada una de las cuatro preguntas tiene su herramienta, y elegir la correcta es el examen.

En el almacén. Es el inspector parado frente al mostrador con su lista.

El oficio pregunta por Araucaria Ferretería SpA, RUT 77884562-8, cuatro cosas.

1. El segmento que tiene registrado hoy.
2. El segmento que tenía al 1 de junio de 2026.
3. El número de modificaciones registradas sobre el padrón y la fecha de cada una.
4. Si alguna de esas modificaciones fue dejada sin efecto después.

Y pide que la respuesta se funde en registros del sistema y no en declaraciones del personal. La tabla no tiene ninguna columna de auditoría, así que las cuatro respuestas tienen que salir de la historia que Iceberg guarda sola.

Paso 2 · Primera pregunta

Celda 2.1 · El segmento de hoy

La pregunta. ¿Qué segmento tiene hoy el contribuyente 77884562-8 ?

Por qué ahora. Es la fácil, y la que se responde con lo que viste en el laboratorio 00.

En el almacén. Es leer el renglón de Araucaria en la página de hoy.

La sentencia, parte por parte.

- `SELECT *` pide todas las columnas.
- `FROM contribuyentes_lab15` lee la tabla como está hoy, sin pedir ninguna versión.
- `WHERE rut = '77884562-8'` se queda con Araucaria.

```
sql
%%sql
-- Celda 2.1
SELECT * FROM contribuyentes_lab15 WHERE rut = '77884562-8'
```

Lo que sale en pantalla.

```
text
rut      razon_social      segmento  region
77884562-8  Araucaria Ferreteria SpA  GRANDE   None
1 fila.
```

Cómo se lee. Hoy Araucaria está en `GRANDE`. Es la respuesta a la primera pregunta.

La cuarta columna, `region`, dice `None`, que es como Python muestra un valor vacío, un `NULL`. La columna se agregó después de que Araucaria entró al padrón, y nadie le puso región.

Paso 3 · Segunda pregunta

Celda 3.1 · El segmento al 1 de junio

La pregunta. ¿Qué segmento tenía Araucaria al 1 de junio de 2026?

Por qué ahora. Aquí ya no sirve consultar la tabla, porque hoy dice otra cosa. Hay que preguntarle por un momento anterior, que es el viaje en el tiempo por fecha del laboratorio 01.

En el almacén. Es pedirle a la libreta la página que mandaba el 1 de junio.

La sentencia, parte por parte.

- Las dos primeras líneas después de `-- Celda 3.1` son comentarios. Avisan que la celda va a fallar y que el fallo es la respuesta.
- `SELECT * FROM contribuyentes_lab15` lee la tabla.
- `TIMESTAMP AS OF '2026-06-01 00:00:00'` pide la tabla como estaba en ese instante, es decir, la última página colgada antes de esa fecha y hora.
- `WHERE rut = '77884562-8'` se queda con Araucaria.

```
sql
%%sql
-- Celda 3.1
-- se espera que falle: el oficio pregunta por una fecha anterior
-- a la primera pagina de la tabla, y esa es la respuesta.
SELECT * FROM contribuyentes_lab15
TIMESTAMP AS OF '2026-06-01 00:00:00'
WHERE rut = '77884562-8'
```

Lo que sale en pantalla.

```
text
Spark rechazó la sentencia:

Cannot find a snapshot older than 2026-06-01T00:00:00+00:00
```

Cómo se lee. Spark rechazó la sentencia, y el mensaje es la respuesta. `Cannot find a snapshot older than 2026-06-01T00:00:00+00:00` quiere decir que no encontró ninguna página anterior a esa fecha. El 1 de junio de 2026 esta tabla todavía no existía.

`2026-06-01T00:00:00+00:00` es la misma fecha escrita en el formato internacional. La `T` separa la fecha de la hora, y `+00:00` dice que es hora UTC.

No hay que buscar una fecha más cómoda ni inventar un dato. Si Contraloría pregunta por un momento en que el registro no existía, lo que corresponde informar es que no existía.

Celda 3.2 · Desde cuándo hay registro

La pregunta. ¿Desde qué momento sí hay registro?

Por qué ahora. Para completar la respuesta a la segunda pregunta hay que decir cuál es el registro más antiguo disponible.

En el almacén. Es mirar la fecha de la primera página de la libreta.

La sentencia, parte por parte.

- `min(committed_at)` busca la hora más antigua en que se colgó una página.
- `AS desde_cuando_hay_registro` le pone nombre a la columna.
- `FROM mi_espacio.contribuyentes_lab15.snapshots` es la vista con la lista de páginas. El nombre tiene tres partes, el espacio, la tabla y la vista.

```
sql
%%sql
-- Celda 3.2
SELECT min(committed_at) AS desde_cuando_hay_registro
FROM mi_espacio.contribuyentes_lab15.snapshots
```

Lo que sale en pantalla.

```
text
desde_cuando_hay_registro
2026-09-25 18:11:18.253
1 fila.
```

Cómo se lee. El registro más antiguo es de las 18:11:18.253 UTC del 25 de septiembre de 2026, las 15:11 en Chile. Es la carga inicial de la celda 0.4, y en esa página Araucaria figuraba como `PEQUENA`. En tu pantalla la fecha va a ser la de tu corrida.

Paso 4 · Tercera pregunta

Celda 4.1 · Las modificaciones y su fecha

La pregunta. ¿Cuántas modificaciones se registraron sobre el padrón, y cuándo fue cada una?

Por qué ahora. El oficio pregunta por el padrón, no por el contribuyente. Lo que hay que entregar es la lista de veces que alguien escribió en la tabla, con su fecha y su tipo, que es la vista del laboratorio 01.

En el almacén. Es la lista de páginas de la libreta, todas las que se escribieron, con su fecha.

La sentencia, parte por parte.

- `SELECT snapshot_id, committed_at, operation` pide el número de cada página, la hora en que se colgó, en UTC, y qué operación fue.
- `FROM mi_espacio.contribuyentes_lab15.snapshots` es la vista con la lista de páginas.
- `ORDER BY committed_at` ordena de la más vieja a la más nueva.

```
sql
%%sql
-- Celda 4.1
SELECT snapshot_id, committed_at, operation
FROM mi_espacio.contribuyentes_lab15.snapshots
ORDER BY committed_at
```

Lo que sale en pantalla.

```
text
snapshot_id      committed_at      operation
5539573441619647012  2026-09-25 18:11:18.253  append
4633828891490677106  2026-09-25 18:11:21.597  overwrite
3537693527222902455  2026-09-25 18:11:22.270  append
3909780055920586729  2026-09-25 18:11:24.383  overwrite
5519598103555517678  2026-09-25 18:11:25.671  overwrite
5 filas.
```

Cómo se lee. Cinco páginas, cinco escrituras sobre el padrón.

- `5539573441619647012`, 18:11:18.253 UTC, `append`. La carga inicial de los diez contribuyentes.
- `4633828891490677106`, 18:11:21.597 UTC, `overwrite`. La reclasificación de Araucaria a `MEDIANA`. Un `UPDATE` sale como `overwrite` porque reescribe el papelito donde está la fila.
- `3537693527222902455`, 18:11:22.270 UTC, `append`. Los tres contribuyentes nuevos.
- `3909780055920586729`, 18:11:24.383 UTC, `overwrite`. La reclasificación de Araucaria a `GRANDE`.
- `5519598103555517678`, 18:11:25.671 UTC, `overwrite`. La baja de Canelo Servicios. Un `DELETE` también reescribe el papelito.

La columna que se agregó no aparece, porque cambiar el esquema no escribe datos y no deja página. La reversión tampoco aparece aquí, porque no escribe una página nueva, solo cambia cuál manda. Eso lo muestra la vista del paso siguiente.

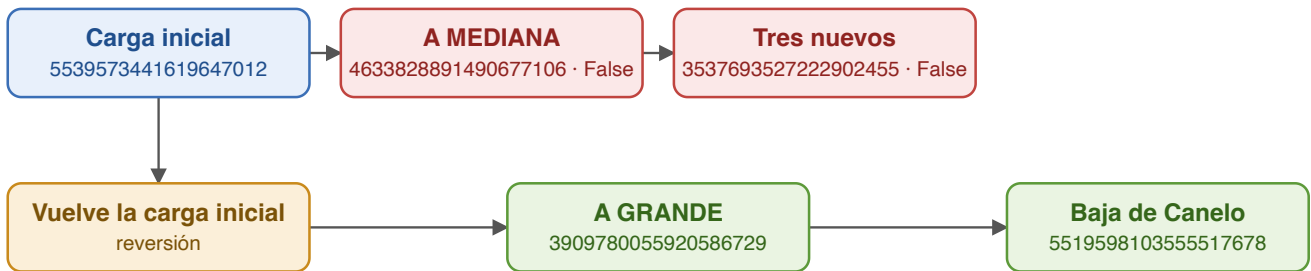
Paso 5 · Cuarta pregunta

Celda 5.1 · Lo que quedó sin efecto

La pregunta. ¿Alguna de esas modificaciones fue dejada sin efecto después?

Por qué ahora. Es la difícil. La lista del paso 4 dice qué se escribió, no si sigue valiendo. Hay otra vista que sí lo dice, con una columna que responde sí o no.

En el almacén. Es el registro de cuándo cada página pasó a mandar. Si una página quedó fuera del camino que llega a la de hoy, es porque alguien volvió atrás.



Carga inicial (azul). Es la primera página, y está en el camino de hoy.

A MEDIANA y **Tres nuevos** (rojo). Son las dos escrituras que vinieron después de la carga inicial y antes de la reversión. Quedaron fuera del camino, con `False`.

Vuelve la carga inicial (amarillo). La reversión hizo que la primera página volviera a mandar. En `.history` aparece de nuevo esa página, con otra hora.

A GRANDE y **Baja de Canelo** (verde). Son las escrituras que vinieron después de la reversión, y forman el camino que llega a la página de hoy.

La sentencia, parte por parte.

- `made_current_at` es cuándo esa página pasó a ser la vigente, en UTC.
- `snapshot_id` es el número de la página.
- `is_current_ancestor` dice si la página está en el camino que llega a la vigente de hoy.
- `FROM mi_espacio.contribuyentes_lab15.history` es la vista con la historia de cuál página mandó.
- `ORDER BY made_current_at` ordena del momento más viejo al más nuevo.

```
sql
%%sql
-- Celda 5.1
SELECT made_current_at, snapshot_id, is_current_ancestor
FROM mi_espacio.contribuyentes_lab15.history
ORDER BY made_current_at
```

Lo que sale en pantalla.

```
text
made_current_at    snapshot_id        is_current_ancestor
2026-09-25 18:11:18.253 5539573441619647012 True
2026-09-25 18:11:21.597 4633828891490677106 False
2026-09-25 18:11:22.270 3537693527222902455 False
2026-09-25 18:11:22.854 5539573441619647012 True
2026-09-25 18:11:24.383 3909780055920586729 True
2026-09-25 18:11:25.671 5519598103555517678 True
6 filas.
```

Cómo se lee. Sí, dos modificaciones quedaron sin efecto. Son las dos filas con `False`.

- 18:11:18.253 UTC, la carga inicial `5539573441619647012`, `True`.
- 18:11:21.597 UTC, la reclasificación a `MEDIANA`, `4633828891490677106`, `False`.
- 18:11:22.270 UTC, los tres nuevos, `3537693527222902455`, `False`.
- 18:11:22.854 UTC, otra vez `5539573441619647012`. Es la reversión, el momento en que la carga inicial volvió a mandar.
- 18:11:24.383 UTC y 18:11:25.671 UTC, la reclasificación a `GRANDE` y la baja, las dos con `True`.

Hay seis filas y no cinco porque la carga inicial pasó a ser la vigente dos veces, al crearse y al volver a ella.

Paso 6 · La respuesta

Este paso no tiene celdas. Es la respuesta al oficio, escrita con los datos de la pantalla.

La pregunta. ¿Qué se le contesta a Contraloría?

Por qué ahora. Con las cuatro respuestas en la mano, falta ponerlas en el formato que pide el oficio.

En el almacén. Es escribirle al inspector lo que dice la libreta, página por página.

1. **Segmento actualmente registrado.** GRANDE .
2. **Segmento al 1 de junio de 2026.** No existe registro, porque la tabla no había sido creada. El registro más antiguo disponible es la carga inicial del padrón, y en ella el contribuyente figuraba como PEQUENA .
3. **Modificaciones sobre el padrón.** Cinco, con la fecha y la hora de cada una según la lista del paso 4.
4. **Modificaciones dejadas sin efecto.** Sí, dos, dejadas sin efecto por una reversión posterior. Figuran en el registro como no vigentes.

Nadie activó un registro de auditoría. No hay una columna de quién modificó ni una tabla de bitácora al lado. Todo lo que acabas de responder venía con la tabla desde su primera escritura.

Preguntas frecuentes

¿Por qué la segunda pregunta se responde con un error?

Porque el error dice exactamente lo que pasó. La tabla no tiene ninguna página anterior al 1 de junio de 2026, así que no hay un estado de esa fecha que mostrar. Informar que el registro no existía es la respuesta correcta y fundada. Inventar un dato o elegir otra fecha sería lo incorrecto.

¿Cuenta la carga inicial como una modificación?

En la lista de `.snapshots` es una escritura más, la primera. El oficio pregunta por las modificaciones registradas sobre el padrón, y la respuesta de la guía cuenta las cinco escrituras, incluida la carga inicial, con su fecha. Si en tu institución la carga inicial no se considera una modificación, la respuesta sería cuatro, y hay que decirlo así en el oficio.

¿Por qué el `ADD COLUMN` no aparece en ninguna de las dos listas?

Porque cambiar el esquema no escribe datos. Cambia la portada de la libreta, el `metadata.json`, pero no cuelga una página nueva. Los cambios de esquema quedan en la vista `.metadata_log_entries`, que no hacía falta para este oficio.

¿Se puede ver cómo estaba la tabla en las páginas que quedaron sin efecto?

Sí. Siguen en la libreta. Con `VERSION AS OF 4633828891490677106` se ve la tabla con Araucaria en `MEDIANA`, aunque esa página ya no esté en el camino de hoy. Mientras nadie las bote con `expire_snapshots`, se pueden leer.

¿Por qué la celda 0.7 es de Python y no SQL?

Porque `CALL` no acepta una consulta como argumento, solo un número escrito. El montaje necesita volver a la primera página sin que el alumno copie un número a mano, así que una celda de Python busca el número y lo pega en la sentencia.

¿Dónde viste cada herramienta?

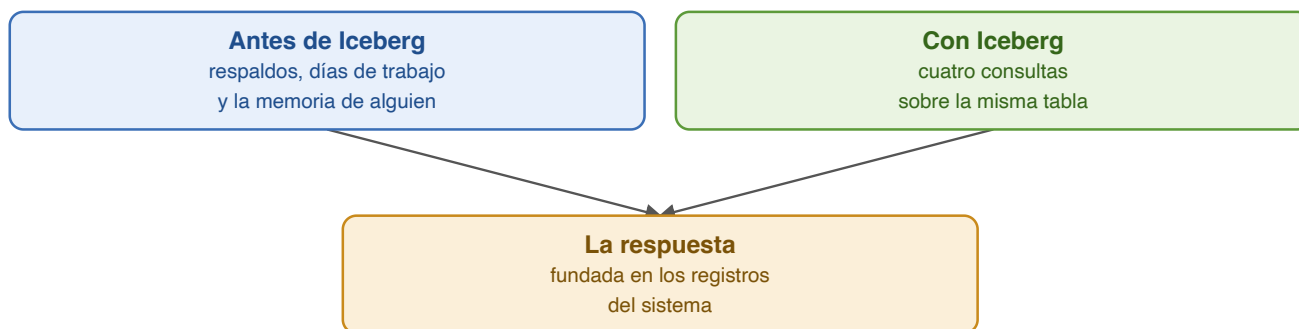
La pregunta	La herramienta	Dónde la viste
Qué segmento tiene hoy	un <code>SELECT</code>	laboratorio 00
Qué segmento tenía el 1 de junio	<code>TIMESTAMP AS OF</code>	laboratorio 01
Cuántas veces cambió	<code>.snapshots</code>	laboratorio 01
Si algo se dejó sin efecto	<code>.history</code> y <code>is_current_ancestor</code>	laboratorio 01

Cierre

La pregunta. ¿Qué te llevas de este laboratorio?

Las cuatro herramientas estaban en los dos primeros laboratorios del curso. Lo que hacía falta no era saber más, era reconocer cuál usar ante una pregunta sin instrucciones. Y una cosa que no estaba en ningún laboratorio, que un error puede ser la respuesta.

En el almacén. El inspector se fue con cuatro respuestas, y todas salieron de la libreta. El almacenero no tuvo que acordarse de nada.



Antes de Iceberg (azul). Responder el oficio significaba buscar en respaldos, reconstruir una tabla de hace meses y firmar que eso era lo que había.

Con Iceberg (verde). Son cuatro consultas sobre la misma tabla, y la historia venía con ella.

La respuesta (amarillo). En los dos casos se contesta el oficio, pero solo en el segundo la respuesta sale de los registros del sistema, como pide Contraloría.

Los números de la solución ejecutada.

Pregunta	Respuesta	De dónde sale
1	GRANDE	celda 2.1
2	no existía registro, el más antiguo es 2026-09-25 18:11:18.253	celdas 3.1 y 3.2
3	cinco escrituras	celda 4.1
4	sí, dos sin efecto	celda 5.1

Eso es lo que separa haber hecho un curso de poder operar la plataforma.